

Locks: Repository Overview

Hongtao Zhang

Mar 23, 2026 · Advisor meeting

Usage-fair delegation locks: code, experiments, stored results, and next algorithmic direction

Today's Agenda

1. **What is in this repo** — workspace structure and execution architecture
2. **What is already implemented** — lock families, workloads, automation, and result pipeline
3. **What the repo already shows** — fairness/locality findings, status gaps, and the March 23 plan

Repo snapshot in one line: this is already a full research workbench, not just a lock library.

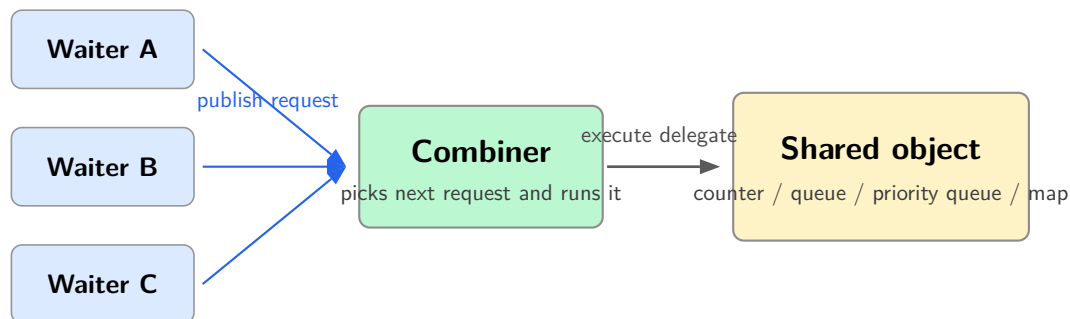
dlock runs experiments, libdlock implements algorithms, c/ supplies reference baselines, visualization/ stores data and plots, and docs//plan/ carry the paper story.

Main tension for the meeting: how much of the next milestone is paper-ready reruns versus an algorithm pivot from FC-PQ toward FC-EW?

The deck is organized around what a new reader actually needs to understand the repository quickly.

What Is a Delegation Lock?

Core idea: waiters do not execute the critical section themselves. They publish a request, and one thread — the **combiner** — executes requests on behalf of everyone.

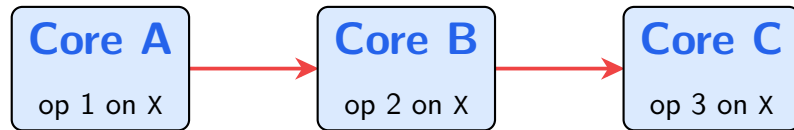


- Each request carries **what operation to perform** and **its arguments**.
- The combiner temporarily acts like a **scheduler** for critical sections.
- Fairness is about **service order and cumulative usage**, not just who touched the lock first.

Why Delegation Can Make Fairness Cheaper

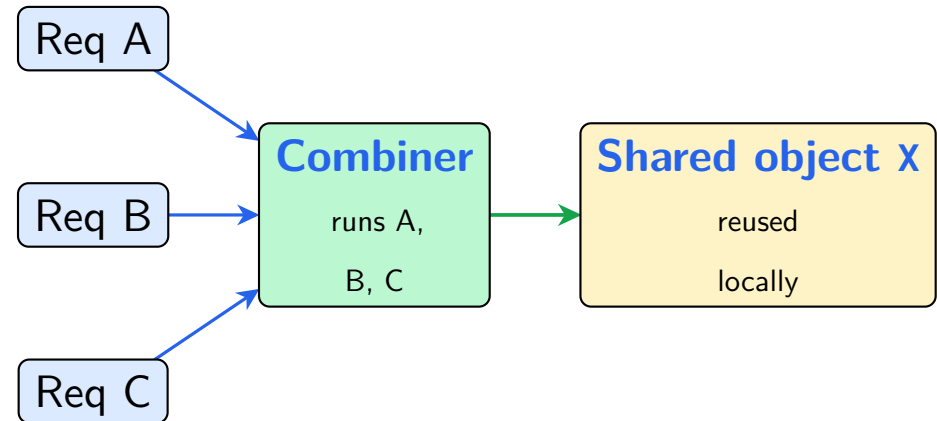
Traditional locks transfer ownership of the critical section from thread to thread. Delegation transfers **requests** while keeping the shared data on one core for a while.

Traditional lock



Each handoff may pull X to a new core.

Delegation



Requests move; the same core can keep touching X.

Question	Traditional queue lock	Delegation lock
What moves?	ownership of the critical section and often the shared cache lines	requests move to the combiner while shared data can stay local
Cost of fairness	fair handoff often increases cross- core migration	reordering inside the combiner can be mostly local
What becomes schedulable?	who acquires next	which request the combiner serves next

How the Main Algorithms Differ

The key fairness signal in this project is **cumulative usage**: how much lock-holding time a thread has already received.

Algorithm	Scheduling rule	Main tradeoff
FC	combiner scans a publication list and serves ready requests	simple and fast, but heterogeneous critical sections can skew usage badly
CC / DSM	same delegation idea with different publication / synchronization structure	alternative engineering points in the same design space

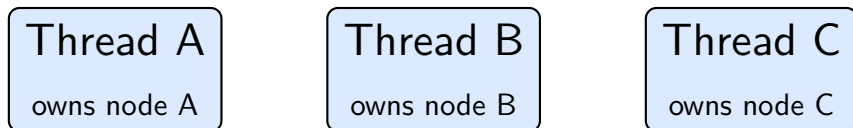
Algorithm	Scheduling rule	Main tradeoff
FC-Ban / CC-Ban	threads that consumed too much recent service are temporarily suppressed	stronger fairness but can stop being work-conserving
FC-PQ	combiner chooses the smallest cumulative-usage waiter next	cleanest fairness story, pays ordering overhead
Traditional fair baselines	MCS/CFL/Ticket/CLH enforce fairer lock handoff directly	fairness often costs more because the shared data still migrates

The March 23 plan argues that FC-PQ should remain the theory anchor, but the next practical algorithm may be an eligibility-window scheduler (FC-EW) rather than another exact heap variant.

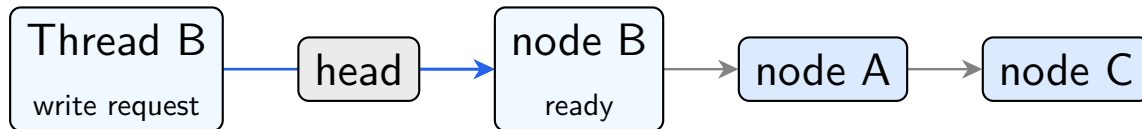
FC: Flat Combining, Phase 1

Core mechanism: FC gives each thread one **persistent publication node**. To request service, a thread writes its operation into its own node, then pushes that node onto the global head list with a CAS loop.

Step 1: each thread owns one persistent node



Step 2: publish by writing your own node, then CAS-push it at head

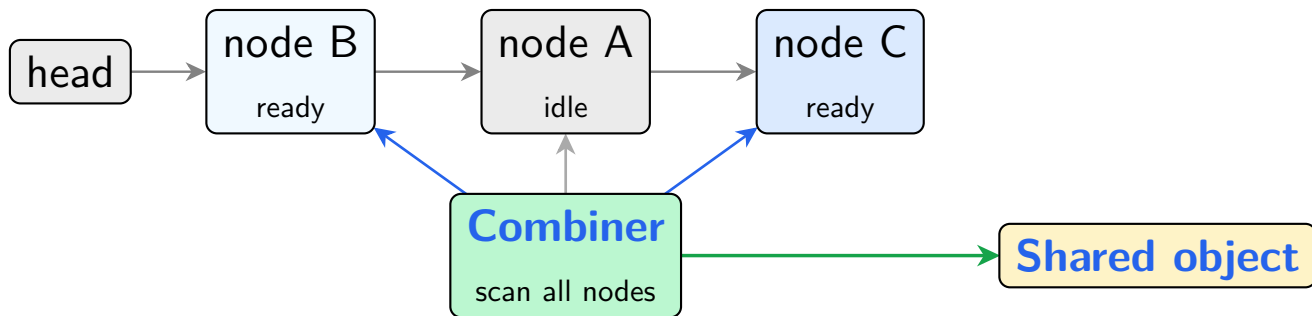


- The request lives in the caller's node; there is no per-request queue object.
- The insertion primitive is a compare-and-swap on the shared head pointer.
- This is the structural difference from CC before the combiner does any work.

FC: Flat Combining, Phase 2

What the combiner pays for: after publication, the combiner walks the whole publication list and executes the nodes that are currently ready. Idle nodes still stay in the structure, so they still get scanned.

Step 3: the combiner scans the whole list and executes ready nodes

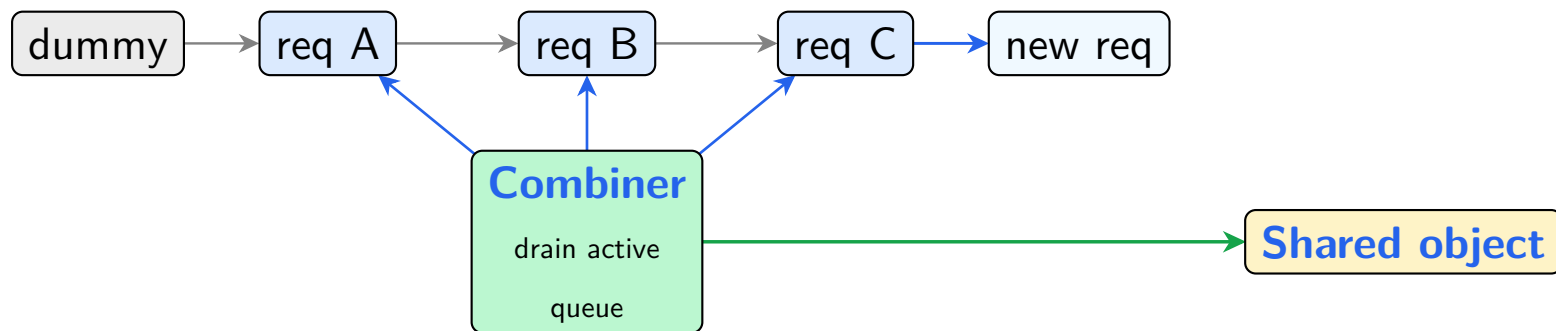


- Strength: publication is simple and a ready node is easy to serve.
- Cost: the combiner scans a per-thread list, including idle nodes.

- Clean contrast with CC: FC pays scan overhead; CC pays queueing overhead.

CC: CCSynch

Fundamental idea: CC is a **per-request queue**. Each request is appended by `tail.swap` rather than a head CAS, so the combiner walks only the active queue rather than scanning thread-owned slots.



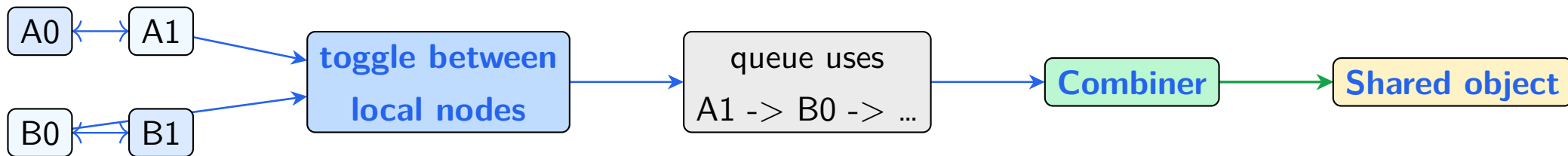
Visual core: the structure contains **only active requests**. Appending a request extends the queue tail, and the combiner drains a consecutive active segment.

- Strength: no need to scan idle thread slots.

- Difference from FC: publication is per-request queueing, not per-thread persistent slots.

DSM

Core mechanism: each thread owns two local nodes and toggles between them, so a new request does not overwrite a node that may still be linked in the queue.

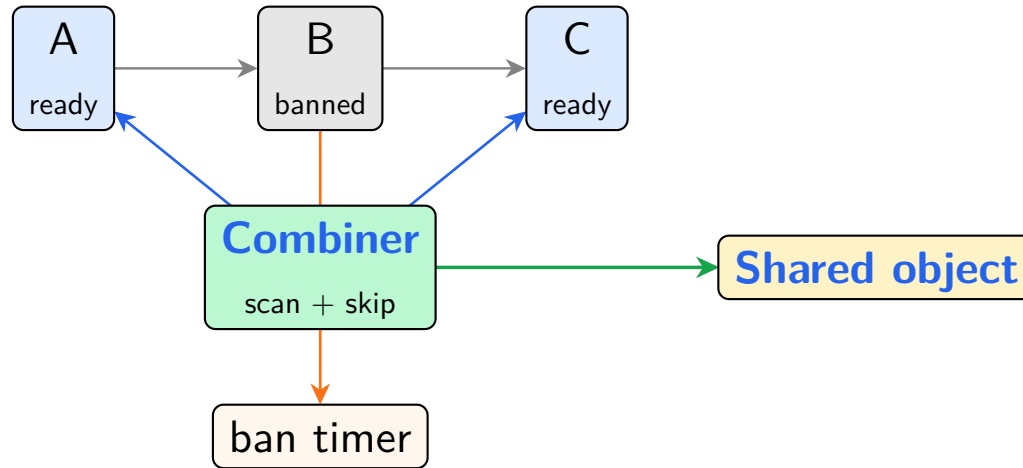


Visual core: each thread alternates between two local nodes, so a new request can be published without clobbering the node already linked in the queue.

- The double-buffering is the distinctive idea here.
- Like FC and CC, it keeps delegation locality but does not solve usage fairness by itself.

FC-Ban

Core mechanism: start from FC's linked-list scan, but each node carries a `banned_until` timestamp; the combiner skips nodes whose penalty window has not expired.

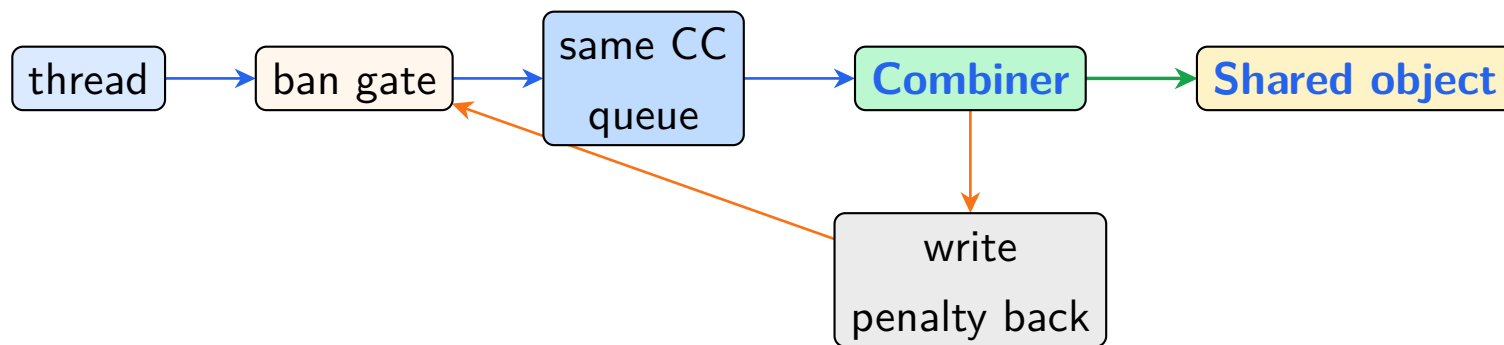


Visual core: same FC list scan, but the combiner refuses to serve a node while its `banned_until` window is still active.

- Benefit: strong correction when one thread has already consumed too much service.
- Cost: can stop being work-conserving because a ready thread may be forced to wait out its ban.

CC-Ban

Fundamental idea: CC-Ban is CC + cooldown. Keep the same queue discipline as CC, but add a `banned_until` gate before a thread can enqueue again, and write back a fresh penalty after service.



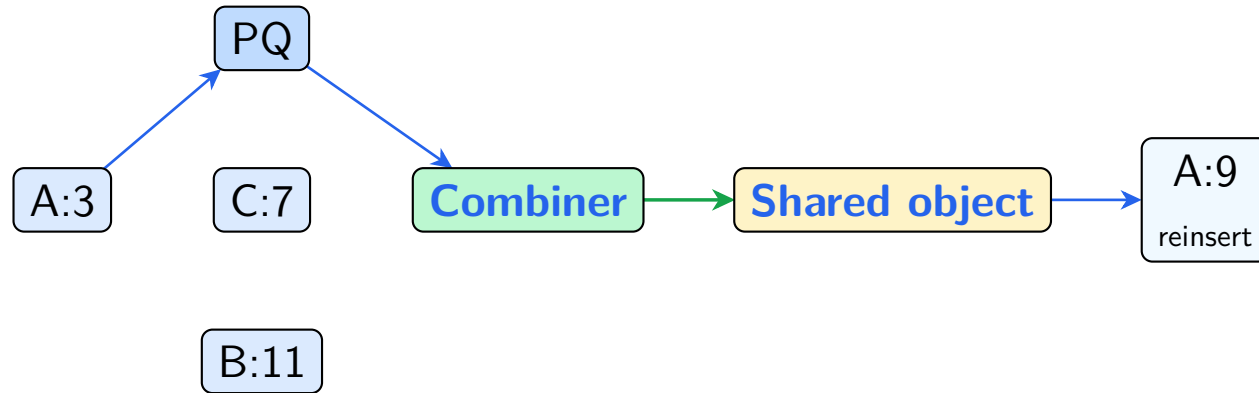
Visual core: do ordinary CC queueing, but only after passing the cooldown gate; after service, the combiner increases that thread's ban window.

- The queue semantics are inherited from CC; the fairness add-on is the cooldown gate.

- Useful as a strict fairness reference point, even if it sacrifices some throughput.

FC-PQ

Core mechanism: drain new waiters into a priority queue keyed by cumulative usage, pop the minimum-usage node, execute it, increase its usage, and reinsert it.



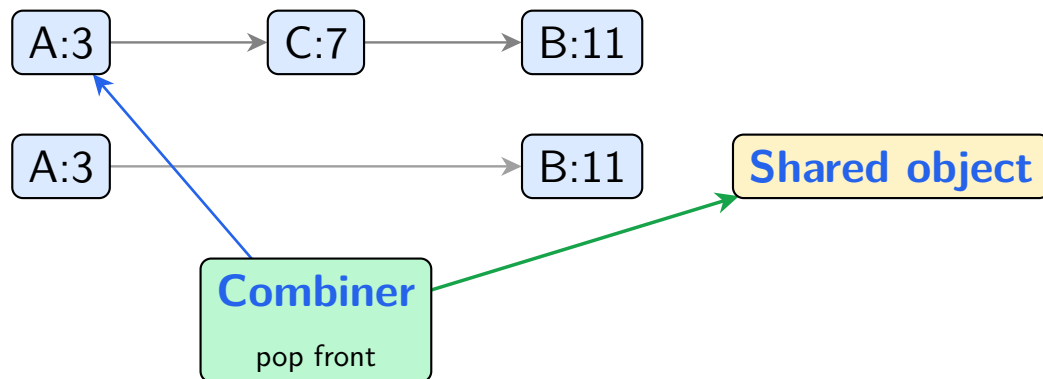
Visual core: always serve the smallest-usage node, then measure its service time and reinsert it with a higher usage value.

- This is the cleanest fairness story in the repo: the combiner is explicitly acting as a min-usage scheduler.

- The research bet is that this ordering cost is still cheaper than fair handoff in traditional locks.

FC-SL

Core mechanism: keep active nodes in a skiplist-style ordered set keyed by cumulative usage; the combiner repeatedly pops the front (minimum) element.



Visual core: maintain an ordered set by usage and repeatedly remove the front (minimum) element; conceptually similar policy to FC-PQ with a different data structure.

- It belongs to the same “fair delegation by ordered service” idea as FC-PQ.

- The difference is the data structure used to maintain order, not the core locality argument.

Workload Matrix

Workload	What it teaches
counter-proportional	heterogeneous critical section ratios, non-CS sweep, fairness and throughput
counter-array	data-footprint and locality experiments with sequential vs random access
fetch-and-multiply	tiny critical sections, optional lock-free baseline
queue	shared queue throughput and response-time studies
priority-queue	shared priority queue benchmarks
hash-map	mixed get/put/scan workload with skew and scanner threads

Common knobs: `--threads`, `--cpus`, `--duration`, `--warmup`, `--trials`, `--stat-response-time`, and workload-specific parameters like `--array-size`, `--random-access`, `--scan-size`, and `--zipf-theta`.

Automation and Measurement

Artifact	Purpose
<code>justfile</code>	common build/run shortcuts for DLock1 and DLock2
<code>experiment.nu</code>	grouped experiment orchestration: ratio sweep, crossover, Pareto, combiner study, perf, delegation-vs-ShflLock
<code>profile.nu</code>	perf stat sweeps over lock variants
<code>BUILD.md</code>	nightly/x86_64 build and test reference
<code>src/benchmark/README.md</code>	execution flow and record schema

Per-thread records already include throughput, hold time, combine time, JFI, normalized share, and optional combiner/waiter latency vectors.

Data and Reporting Assets

- **Output corpus:** 36 top-level result directories under `visualization/output/`
- **Stored runs:** 918 Arrow files already in-tree
- **Python analysis:** `plot_pareto.py`, `read_arrow_tables.py`, `analyze_combiner.py`, `summarize_latency.py`, `throughput_table.py`, and related scripts
- **Writing artifacts:** Typst proposal, report, earlier meeting notes, and this new presentation

The repo is set up to go from algorithm code to paper figure without leaving the tree: run benchmarks, emit Arrow, aggregate with Python, and present with Typst.

What the Existing Repo Results Already Say

These numbers come from docs/EXPERIMENT_RESULTS_DEMO.md and the experiment-plan narrative already checked into the repository.

Observation	Representative value	Interpretation
FC_PQ_BHeap fairness	0.9982 JFI	at 64 threads and 1:100 CS ratio in the demo table, showing that fair delegation can stay near unit fairness
CCBan fairness	> 0.99 JFI	across the reported 64-thread ratio sweep, making it the fairness champion in the checked-in summary

Observation	Representative value	Interpretation
Mutex collapse	0.016 JFI	at 64 threads and 1:100 ratio, showing how badly traditional unfair serialization can skew usage
FC / MCS locality gap	2.2×	at 32 KiB random counter-array footprint, reinforcing the data-migration story
USCL tradeoff	fair but slower	docs describe it as consistently fair but roughly 3–5× below delegation throughput

The repo's intended paper claim is already visible in the stored material: the cost of adding fairness to delegation should be much smaller than the cost of adding fairness to queue locks.

What Looks Done

- `TOD0.md` marks MCS, CFL, Ticket, CLH, response-time CDF export, newcomer initialization, starvation counters, and prefetching as completed.
- `experiment.nu` already scripts the major paper-shaping groups: benefits, tradeoff, combiner study, factor analysis, and perf validation.
- `visualization/output/` already contains enough breadth to synthesize an interim story before running new jobs.

The repository has already crossed from “prototype implementation” into “evaluation platform with paper infrastructure.”

What Still Needs Reconciliation

- STATUS_REPORT.md is older and still flags waiting-thread accounting problems in FC-Ban and CC-Ban.
- Several final reruns in TODO.md remain unchecked, especially long-duration paper-ready collections.
- Environment capture and result hygiene are still weaker than the algorithm work.

Practical issue for the meeting: the code, the stored results, and the status documents are no longer perfectly synchronized. Before a paper push, the repo needs one documentation pass that says exactly what is true today.

March 23 Direction: Keep FC-PQ, Consider FC-EW

Recommendation in [plan/2026-03-23/algorithm-improvement-plan.md](#): keep FC-PQ as the correctness and theory anchor, but stop spending time on more heap variants and consider FC-EW as the practical next algorithm.

- **Why change?** Exact min-usage scheduling is analytically clean, but it pays ordering cost even when many waiters are already “fair enough.”
- **FC-EW idea:** replace exact global min-ordering with an eligibility window, so the combiner chooses from waiters whose cumulative usage is within delta of the current minimum.
- **Expected benefit:** preserve strong usage-fairness while reducing combiner-side scheduling overhead and giving more room to address combiner asymmetry.

Immediate decision point: do we spend the next iteration finishing the paper-ready FC-PQ story, or pivot early and make FC-EW the centerpiece before the evaluation is locked in?

This repo already contains the paper's spine.

Algorithms, baselines, workloads, automation, stored results, analysis scripts, and writing artifacts are all here.

The main remaining work is not “build the project” — it is choosing and tightening the final research story.